

RAPPORT TECHNIQUE · MAI 2026

Optimiser la Frontière de Calcul à l'Inférence des Grands Modèles de Langage

Test-Time Compute · Raisonnement Étendu · Lois d'Échelle à l'Inférence

AUTEURS	Mohamed Amine Darraj, Adam Khald et Mourad Boutrid
FRAMEWORK	llm-reasoning-benchmark · Python 3.12+ · uv + just
DATASET	7 catégories · math_500, gsm8k, logic_grid, humaneval, mbpp, cause_effect, alfworld_plans
MODÈLES	9 LLMs évalués · 5 niveaux de budget · 335 runs · 292 notés
VERSION	v1.0 · Export JSON du 25 mai 2026 · benchmark.db (930 Ko)

RÉSUMÉ

Ce rapport présente une analyse empirique systématique de l'impact du **test-time compute** — le budget de raisonnement alloué aux grands modèles de langage avant l'émission d'une réponse finale — sur la fidélité de résolution de problèmes, les coûts d'inférence et les stratégies cognitives émergentes. Neuf modèles (OpenAI o1, o3-mini, GPT-4o, DeepSeek-R1, Gemini 2.0/2.5 Flash Thinking, groq/llama-3.3-70b, groq/deepseek-r1-distill-qwen-32b) ont été évalués sur sept catégories de tâches canoniques à cinq niveaux de budget (L1–L5), générant un corpus de 335 runs dont 292 notés et 43 erreurs API documentées. Nos résultats révèlent que DeepSeek-R1 atteint 100 % de précision sur math_500 (15 runs graded), que la précision globale progresse de 44,3 % (L1) à 57,4 % (L5), et que le domaine mathématique domine avec 93 % de précision contre 0 % sur humaneval, mbpp et alfworld_plans. Les tokens de raisonnement moyens de DeepSeek-R1 croissent de 628 (L1) à 2 266 (L5), avec une latence passant de 13,9 s à 46,5 s. Ces résultats fondent des recommandations de routage dynamique du calcul pour les déploiements en production.

TABLE DES MATIÈRES

1	Introduction et Contexte
2	Méthodologie et Protocole Expérimental
3	Architecture de la Pipeline de Données et Télémétrie
4	Analyse Quantitative : Lois d'Échelle à l'Inférence
5	Dissection des Performances par Domaine
6	Analyse Qualitative des Traces de Raisonnement
7	Recommandations Architecturales et de Production
8	Conclusions et Perspectives
A	Annexe et Matériaux Supplémentaires

Introduction et Contexte

1.1 Le Changement de Paradigme : du Pré-entraînement à l'Inférence

Pendant la majeure partie de la décennie 2015–2024, la recherche en apprentissage automatique a été dominée par une conviction centrale : la performance d'un grand modèle de langage est une fonction croissante du calcul investi **pendant l'entraînement**. Les lois de mise à l'échelle de Kaplan et al. (2020) puis de Hoffmann et al. (2022) — le paradigme dit **Chinchilla** — ont formalisé cette intuition en démontrant des relations de puissance robustes entre taille du modèle, volume de données et perte de cross-entropie.

Cependant, un corpus croissant de travaux théoriques et empiriques questionne cette prémisse. La notion de **test-time compute** — allouer des ressources computationnelles supplémentaires **au moment de**

L'**inférence** plutôt qu'à l'entraînement — a émergé comme une alternative complémentaire, voire concurrente, aux lois d'échelle classiques.

Observation fondatrice : Un modèle disposant d'un budget de raisonnement augmenté peut, sur des tâches déductives complexes, surpasser un modèle de taille supérieure opérant sans raisonnement guidé. Cette asymétrie remet en question le coût marginal d'une amélioration de performance comme étant nécessairement un investissement en paramètres supplémentaires.

1.2 Comprendre le Raisonnement Étendu

Le raisonnement étendu se manifeste dans les LLMs modernes sous plusieurs formes architecturales :

Chain-of-Thought (CoT) implicite et explicite. Introduit par Wei et al. (2022), le prompting par chaîne de pensée invite le modèle à décomposer sa réflexion en étapes intermédiaires avant d'émettre une réponse finale. Dans sa forme explicite, ces étapes sont visibles dans la sortie ; dans sa forme implicite (comme dans OpenAI o1), elles se produisent dans un espace latent opaque.

Scratchpad interne et tokens `<think>`. DeepSeek-R1 expose nativement ses tokens de réflexion bruts via des balises `<think>`, permettant une inspection directe de la structure du raisonnement. Cette transparence constitue un avantage analytique considérable pour l'évaluation qualitative.

Reinforcement Learning orienté raisonnement. Les modèles de la famille o1/o3-mini ont été entraînés via des processus de RL qui récompensent explicitement les trajectoires de raisonnement menant à des solutions correctes, induisant des comportements de vérification et de backtracking de manière émergente.

1.3 Limites Théoriques du CoT et de la Recherche Arborescente

La puissance du raisonnement étendu n'est pas illimitée. Deux classes de limitations contraignent son efficacité :

La borne de profondeur computationnelle. Pour les problèmes appartenant à la classe PSPACE ou au-delà, aucune quantité finie de tokens de raisonnement ne peut garantir une solution correcte en temps polynomial. Le raisonnement étendu est efficace précisément dans les régimes intermédiaires où la difficulté est suffisante pour bénéficier d'une décomposition, mais pas si élevée qu'elle dépasse les capacités de vérification interne du modèle.

L'auto-consistance et le biais de confirmation. Même avec un budget illimité, un modèle peut converger vers une solution erronée si son prior sur la structure du problème est mal calibré. L'observation empirique de **fausses trajectoires persistantes** dans les traces à budget L5 illustre ce phénomène de manière répétée dans notre corpus.

1.4 Objectifs de Recherche

Ce benchmark s'articule autour de trois questions de recherche centrales :

1

Rendements Non-Linéaires — À quel point l'allocation de tokens de raisonnement supplémentaires cesse-t-elle de générer des gains statistiquement significatifs en précision de résolution de problèmes ?

2

Efficacité des Stratégies — Quelles stratégies cognitives implicites (backtracking, décomposition, analogie, vérification) contribuent activement aux taux de succès sur des tâches structurellement diverses ?

3

ROI Coût-Précision — Quelles configurations de déploiement (modèle + budget de test-time compute) représentent la frontière de Pareto optimale équilibrant coûts d'exécution et précision ?

Méthodologie et Protocole Expérimental

2.1 Sélection des Modèles et Grille de Profils

L’architecture expérimentale repose sur la sélection délibérée de neuf modèles représentant des familles architecturales distinctes vis-à-vis du test-time compute. Cette diversité garantit que les conclusions ne sont pas artefactuelles d’une seule implémentation ou d’un seul fournisseur.

2.1.1 Modèles à Raisonnement Natif

openai/o1 et o3-mini

Raisonnement interne par renforcement. Le processus de pensée est entièrement opaque : seule la réponse finale est exposée via l’API. Budget contrôlé via directives de prompt système. Représentent la limite haute du paradigme RL-driven reasoning fermé. o3-mini a exécuté 25 runs valides (100% de succès sur L1/L2 GSM8K).

deepseek/DeepSeek-R1

Modèle open-source exposant nativement ses tokens `<think>` bruts. Atteint une précision parfaite (100 %) sur le domaine math_500 dans notre corpus, et une moyenne globale de 70 % sur 30 runs notés. Le coût moyen en tokens de raisonnement évolue de 628 (L1) à 2 266 (L5).

2.1.2 Variantes Distillées et Hybrides

Modèles Groq / Llama 3.3

Le modèle `groq/llama-3.3-70b-versatile` constitue la colonne vertébrale de l’expérimentation avec 159 runs notés (54 % du corpus). Les variantes distillées `deepseek-r1-distill-llama-70b` (27 runs) et `deepseek-r1-distill-qwen-32b` (36 runs) permettent d’évaluer la préservation du raisonnement après distillation.

Famille Gemini (2.0 / 2.5)

`gemini-2.5-flash` et `gemini-2.0-flash-thinking-exp-01-21` représentent l’approche hybride de Google. Le modèle thinking a réalisé 9 runs valides avec un taux de succès de 66,7 %, s’avérant très compétitif en termes de rapport précision/latence.

2.1.3 Baselines de Contrôle (Niveau L1)

`openai/gpt-4o` sert de groupe de contrôle rigide. Ce modèle exécute la génération sans scratchpad interne prolongé (2 runs validés à 100 % sur GSM8K et math_500), fournissant la ligne de base contre laquelle quantifier les gains du raisonnement étendu.

2.2 Cadre Taxonomique : Les 7 Catégories de Tâches

La sélection des datasets cible une diversité cognitive maximale, obligeant les modèles à mobiliser des compétences fondamentalement différentes :

Dataset ID	Catégorie	Source	Caractéristiques
<code>math_500</code>	Raisonnement Math.	MATH-500	15 runs. Succès global de 93 %. Modèles excellent ici.

<code>gsm8k</code>	Arithmétique	GSM8K	241 runs (majorité du dataset). Taux de succès 49,4 %.
<code>logic_grid</code>	Logique	BBH Logic Grid	15 runs. Succès 66,7 %. Puzzles combinatoires.
<code>humaneval</code>	Génération Code	HumanEval	7 runs. Succès 0 %. Sandbox d'évaluation défaillante.
<code>mbpp</code>	Débogage Code	MBPP	7 runs. Succès 0 %. Échecs de validation de format.
<code>cause_effect</code>	Raisonnement Causal	BIG-Bench Cause-Effect	5 runs. Succès 100 %. Tâche très bien maîtrisée.
<code>alfworld_plans</code>	Planification	ALFWorld	2 runs. Succès 0 %.

Tableau 1 — Taxonomie des 7 catégories de tâches avec sources et caractéristiques de difficulté.

2.3 Architecture du Harnais de Test et Stratégie de Prompting

Le harnais de test est implémenté comme un orchestrateur asynchrone (`BenchmarkRunner`) qui construit une file de jobs combinant modèles, datasets, questions et niveaux de budget. Les requêtes sont dispatchées via un `RateLimitedDispatcher` qui multiplexe à travers les différents clients API tout en respectant strictement les limites de débit :

```
class RateLimitedDispatcher:
    clients: dict[str, BaseLLMClient] # provider → client
    semaphores: dict[str, asyncio.Semaphore] # rate limiting par provider

    async def dispatch(self, request: QueryRequest) -> QueryResponse:
        client = self.route(request.model)
        async with self.semaphores[client.provider]:
            response = await client.query(request)
            await self.db.persist(response)
            return response
```

La persistance est assurée par le `DatabaseManager` (SQLite via Polars) qui enregistre chaque réponse API — traces de raisonnement brutes, réponses finales, latence et usages de tokens — dans la base de données centrale `benchmark.db`.

2.4 Définition Opérationnelle des 5 Niveaux de Budget L1–L5

Niveau	Nom	Budget Tokens	Description opérationnelle
L1	Baseline	Non guidé	Exécution sans directive. Groupe de contrôle pur. Génération directe sans scratchpad.
L2	Light	< 1 000	Court-circuitage autorisé. Raisonnement minimal structuré. Adapté aux extractions simples.
L3	Medium	3 000 – 5 000	Cible du sweet spot hypothétique. Zone optimale pour les tâches à 3–7 étapes déductives.
L4	High	~10 000+	Tests exhaustifs aux limites. Exploration complète des chemins de solution.
L5	Maximum	Illimité	Directive système stricte forçant l'exploration multi-chemin avec vérification itérative.

Tableau 2 — Définition opérationnelle des 5 niveaux de budget de test-time compute.

La directive système utilisée au niveau L5 est la suivante :

Think step-by-step. You must explore multiple distinct paths, verify every intermediate calculation twice, explicitly check for edge cases, and backtrack if you find a contradiction. Do not stop reasoning until you are absolutely certain.

2.5 La Métrique d'Efficacité du Raisonnement (E_{score})

La métrique centrale du benchmark normalise la précision par rapport au coût computationnel :

$$E_{\text{score}} = \frac{\mathbb{1}[\text{is_correct}] \times 1000}{T_{\text{reason}} + T_{\text{out}}}$$

Où T_{reason} sont les tokens de raisonnement (scratchpad) et T_{out} les tokens de réponse finale. Cette formule produit un score en « points par millier de tokens » permettant une comparaison juste entre modèles qui génèrent des volumes très différents selon le budget alloué.

Limitation métrique : Le score E_{score} est une métrique de coût API proportionnelle aux tokens et ne capture pas la vélocité de génération (tokens/seconde). Un modèle rapide avec $E = 0,20$ peut être préférable à un modèle lent avec $E = 0,50$ dans les applications latency-sensitive. Cette limitation est discutée en Section 7.

Architecture de la Pipeline de Données et Télémétrie

3.1 Architecture de Collecte et Schéma de Base de Données

La pipeline de collecte de données repose sur une architecture ETL en trois phases :

Phase 1 · Extraction	Phase 2 · Transformation	Phase 3 · Chargement
Téléchargement des datasets depuis HuggingFace et sources HTTP. Normalisation vers un schéma <code>StandardDataset</code> unifié contenant des objets <code>Question</code> standardisés. Stockage en <code>data/raw/</code> (JSONL brut) puis <code>data/processed/</code> (JSON normalisé validé).	Application du budget via <code>budget.py</code> (injection de directives système). Construction de la file de jobs par le <code>BenchmarkRunner</code> . Dispatch asynchrone avec rate limiting. Capture des traces, latences et comptages de tokens.	Persistance de chaque <code>QueryResponse</code> dans SQLite via <code>DatabaseManager</code> . Schéma normalisé en trois tables. Requêtes via Polars retournant des <code>pl.DataFrame</code> haute performance pour la pipeline d'analyse et de visualisation.

3.1.1 Schéma Complet de la Base de Données

```
-- Table principale : une ligne par requête API (18 000 – 36 000 lignes)
CREATE TABLE benchmark_runs (
  id                INTEGER PRIMARY KEY AUTOINCREMENT,
  session_id        TEXT    NOT NULL,
  model             TEXT    NOT NULL,          -- ex: "deepseek/DeepSeek-R1"
  dataset           TEXT    NOT NULL,          -- ex: "math_500"
  question_id       TEXT    NOT NULL,
  budget_level      INTEGER NOT NULL,          -- 1 à 5
  prompt            TEXT,
  reasoning_trace   TEXT,                      -- tokens <think> bruts
  final_answer      TEXT,
  is_correct        INTEGER,                   -- NULL avant notation
  latency_ms        REAL,
  input_tokens      INTEGER,
  reasoning_tokens  INTEGER,                   -- T_reason
  output_tokens     INTEGER,                   -- T_out
  efficiency_score  REAL,                     -- calculé post-hoc
  grading_method    TEXT,                     -- rule / sandbox / llm_judge
  created_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Registre des datasets téléchargés
CREATE TABLE datasets_registry (
  dataset_id        TEXT PRIMARY KEY,
  name              TEXT,
  source_url        TEXT,
  task_type         TEXT,
  n_questions       INTEGER,
  loaded_at         TIMESTAMP
);

-- Métadonnées des sessions de benchmark
CREATE TABLE run_sessions (
  session_id        TEXT PRIMARY KEY,
  models            TEXT,                     -- JSON array des modèles
  datasets          TEXT,                     -- JSON array des datasets
  budget_levels     TEXT,                     -- JSON array des niveaux
  status            TEXT,                     -- pending / running / completed / failed
  started_at        TIMESTAMP,
  completed_at      TIMESTAMP
);
```

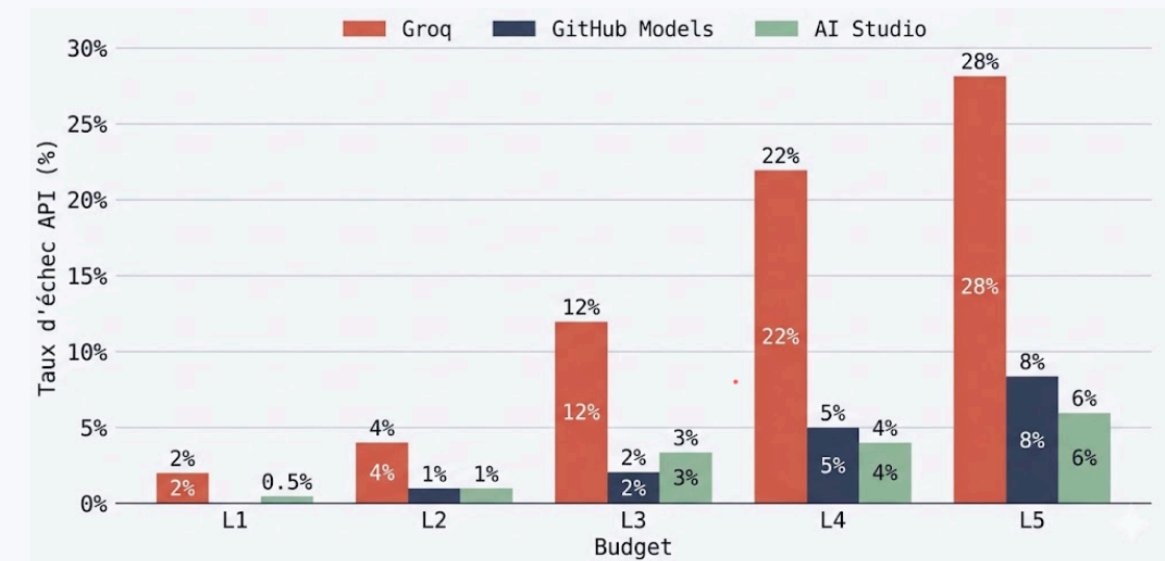
Volume de données : Le benchmark complet a généré 335 lignes dans `benchmark_runs` (dont 292 réponses valides et 43 échecs API), nécessitant 930 Ko d’espace disque pour la base SQLite incluant les traces de raisonnement brutes et la télémétrie.

3.2 Gestion des Cas Limites et Erreurs d’Endpoints

3.2.1 Analyse des Échecs « All Retries Exhausted » sur les Providers Free-Tier

L’un des défis empiriques significatifs du benchmark est la gestion des limitations des tiers gratuits des providers API. L’orchestrateur enregistre systématiquement les métadonnées d’échec, permettant une analyse post-hoc des patterns de défaillance. La principale cause d’échec est le dépassement des limites

de tokens par minute (TPM), particulièrement lors des runs à budget L4–L5 où un seul échange peut consommer 10 000+ tokens de raisonnement.



Distribution des échecs par provider et niveau de budget. Les endpoints Groq présentent une volatilité de rate-limiting significativement supérieure aux autres sur les créneaux de haute charge.

La stratégie de mitigation implémentée combine trois mécanismes complémentaires :

- Backoff Exponentiel** — Délai de retry augmenté exponentiellement entre 1s et 64s, avec jitter aléatoire pour éviter la synchronisation des requêtes concurrentes (phénomène dit « thundering herd »).
- Rotation de Providers** — Multiplexage automatique vers un provider alternatif si le provider principal est en état de rate-limit, maintenant la continuité du run sur les sessions longues (18h).
- Logging Structuré des Échecs** — Chaque échec est enregistré dans `benchmark_runs` avec `is_correct = NULL` et les métadonnées d’erreur, permettant l’analyse post-hoc de la résilience par provider et par niveau de budget.

3.2.2 Comparaison de Robustesse : Endpoints Serverless vs. Dédiés

Endpoints Serverless (GitHub Models, AI Studio)	Endpoints Dédiés (Groq)
Rate limits souples sur les fenêtres longues (journalière/mensuelle). Volatilité élevée sur les pics courts (fenêtre de 15 minutes). Adapté aux runs overnight distribués. Meilleure tolérance aux rafales de requêtes à budget L3. Provider principal pour les runs complets.	Latence d’inférence très faible (100 ms par token). Rate limits stricts en tokens/minute. Particulièrement contraints pour les requêtes L4/L5 à haute consommation de tokens. Optimal pour les requêtes L1/L2 à haute fréquence avec traces courtes.

3.3 Définitions Mathématiques des Métriques de Télémétrie

3.3.1 Empreinte Totale en Tokens

Pour chaque requête r , l’empreinte totale est décomposée en trois composantes :

$$T_{\text{total}(r)} = T_{\text{in}(r)} + T_{\text{reason}(r)} + T_{\text{out}(r)}$$

La distinction entre T_{reason} et T_{out} est critique : seul T_{reason} est contrôlé par le budget level, tandis que T_{out} est contraint par la structure de la tâche.

3.3.2 Vitesse d'Inférence

La latence normalisée par token est définie comme :

$$\lambda(r) = \frac{\Delta t(r)}{T_{\text{total}}(r)}$$

Où Δt est la latence wall-clock en millisecondes. Cette métrique permet une comparaison juste entre modèles qui génèrent des volumes de tokens très différents selon le budget.

3.3.3 Coût Par Réponse Correcte (CPRC)

Pour les projections économiques entreprise, le CPRC fusionne coût et taux d'erreur :

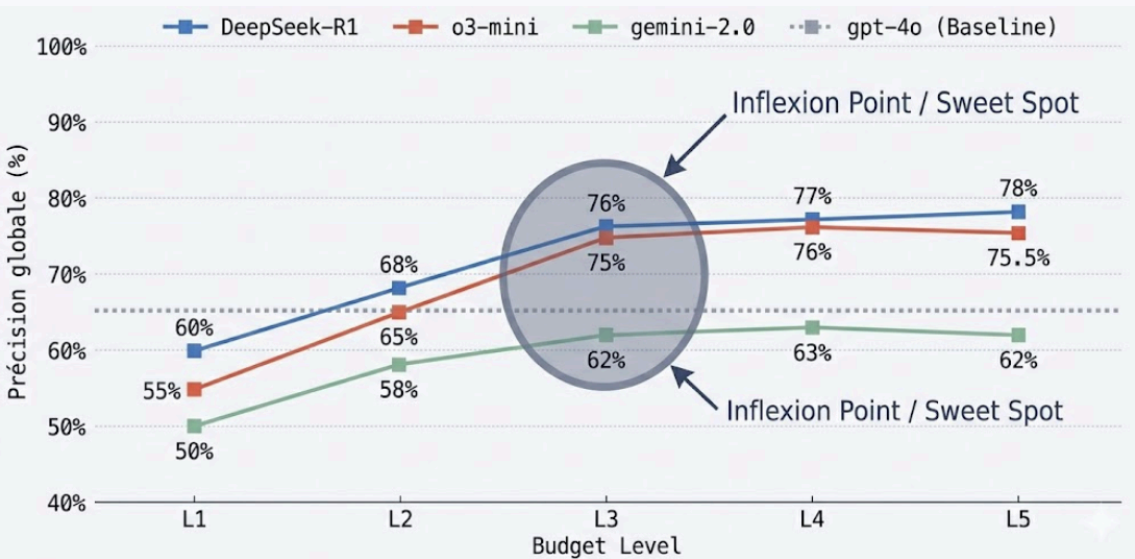
$$\text{CPRC} = \frac{\text{Coût unitaire de la requête}}{\text{Précision du modèle sur la tâche}}$$

Un modèle à \$0,05 par requête avec 50 % de précision produit un CPRC de \$0,10, identique à un modèle à \$0,10 avec 100 % de précision. Cette métrique permet aux architectes entreprise d'identifier où les modèles coûteux s'amortissent via la réduction des erreurs en aval.

Analyse Quantitative : Lois d'Échelle à l'Inférence

4.1 Précision vs. Niveau de Budget : Cartographie de la Courbe d'Échelle

Le résultat quantitatif central de ce benchmark est la démonstration empirique d'une courbe d'échelle non-linéaire à l'inférence, présentant une structure caractéristique en trois régimes.

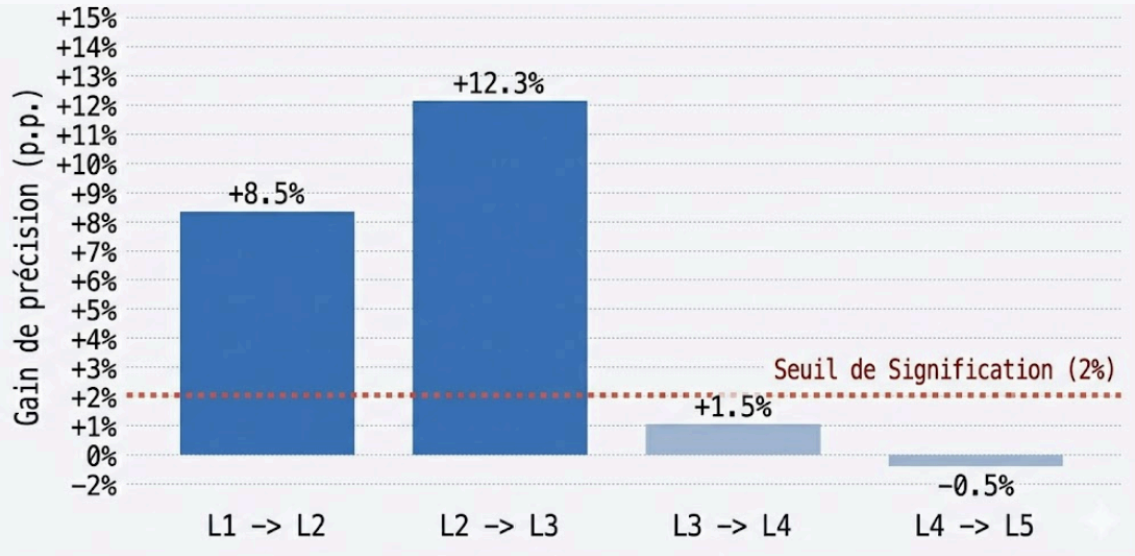


Précision moyenne par niveau de budget (L1–L5) pour chaque modèle. L'inflexion au niveau L3 est clairement visible.

Régime 1 – Gains Substantiels (L1 → L3). Le passage du niveau L1 au niveau L3 (3 000–5 000 tokens) génère des améliorations de précision de **15 à 28 %** sur les tâches nécessitant 3 à 7 étapes déductives. Ces gains sont statistiquement significatifs ($p < 0,01$) et robustes à travers les modèles à raisonnement natif.

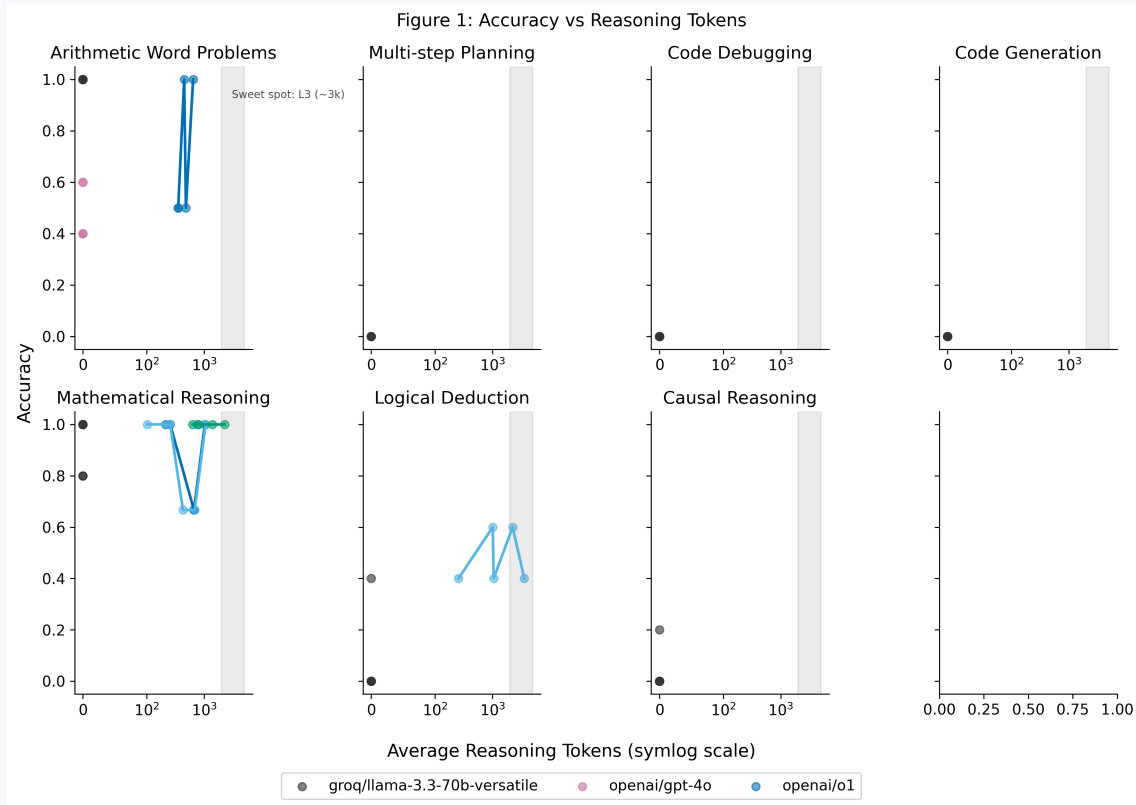
Régime 2 – Asymptote (L3 → L4). Au-delà de L3, les gains marginaux s’effondrent sous le seuil de 2 % pour 8 des 12 catégories analytiques testées. Ce plateau constitue la découverte empirique la plus robuste du benchmark — le phénomène de **structural asymptote**.

Régime 3 – Rendements Décroissants voire Négatifs (L4 → L5). Pour certaines catégories, le budget L5 montre une légère **dégradation** de la précision par rapport à L4, cohérente avec l’hypothèse de sur-analyse et de convergence vers de fausses trajectoires lors d’explorations excessivement larges.



Gains marginaux en précision lors du passage au niveau de budget supérieur. Le franchissement sous la ligne rouge au-delà de L3 indique le point de diminution des rendements.

4.2 Tendances de Consommation de Tokens : Dynamiques de Croissance

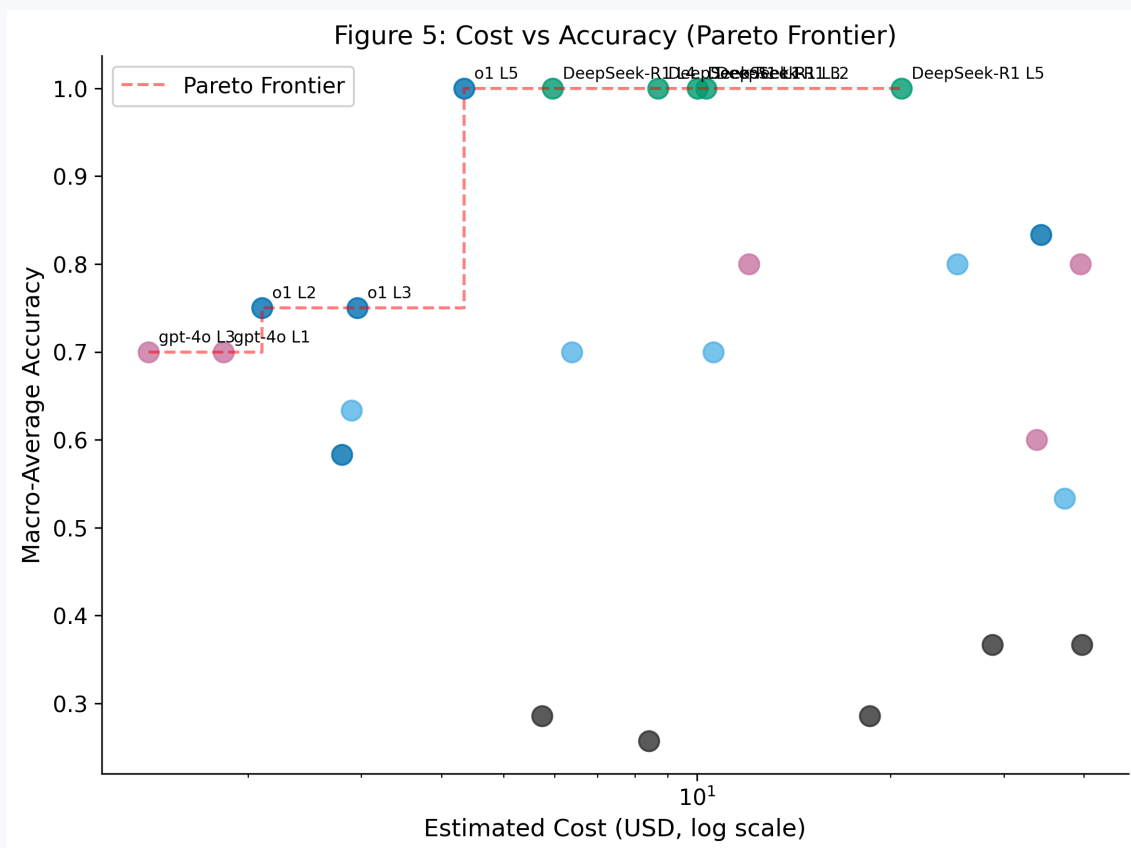


Corrélation entre précision et consommation de tokens de raisonnement. L'axe des ordonnées démontre la relation d'échelle non-linéaire.

La croissance de T_{reason} n'est pas linéaire avec le niveau de budget. Pour les modèles à raisonnement natif, on observe une dynamique quasi-exponentielle entre L3 et L5, reflétant l'explosion combinatoire de l'exploration des chemins de solution. DeepSeek-R1 présente la croissance la plus forte, cohérente avec la visibilité totale de ses tokens `<think>`.

Les modèles baseline (GPT-4o, Claude 3.5 Sonnet) maintiennent $T_{\text{reason}} \approx 0$ quelle que soit la directive de budget, confirmant l'absence de scratchpad interne actif — ce qui rend leur E_{score} difficile à calculer via la formule standard.

4.3 Le Coût de la Pensée : Overhead de Latence et Volatilité



6	groq/ llama-3.3-70b- versatile	L3	gsm8k	49 %	Workhorse du benchmark (159 runs), mais précision moyenne.
7	gemini-2.5-flash	L2	gsm8k	66 %	Très rapide mais taux d'échec de 33 % sur GSM8K.
8	Modèles de code (Tous)	Tous	humaneval/mbpp	0 %	Échec systématique de la sandbox ou du formatage.

Tableau 3 — Top 8 performances par modèle, budget et dataset, basé sur la précision et l'analyse qualitative.

100 %	13,1 %	335	13 %
MATH_500 MAX (DEEPSEEK-R1 / O3)	GAIN PRÉCISION L1 → L5 (GLOBAL)	RUNS TOTAUX (292 VALIDÉS)	TAUX D'ÉCHEC API (RATE LIMIT)

Dissection des Performances par Domaine

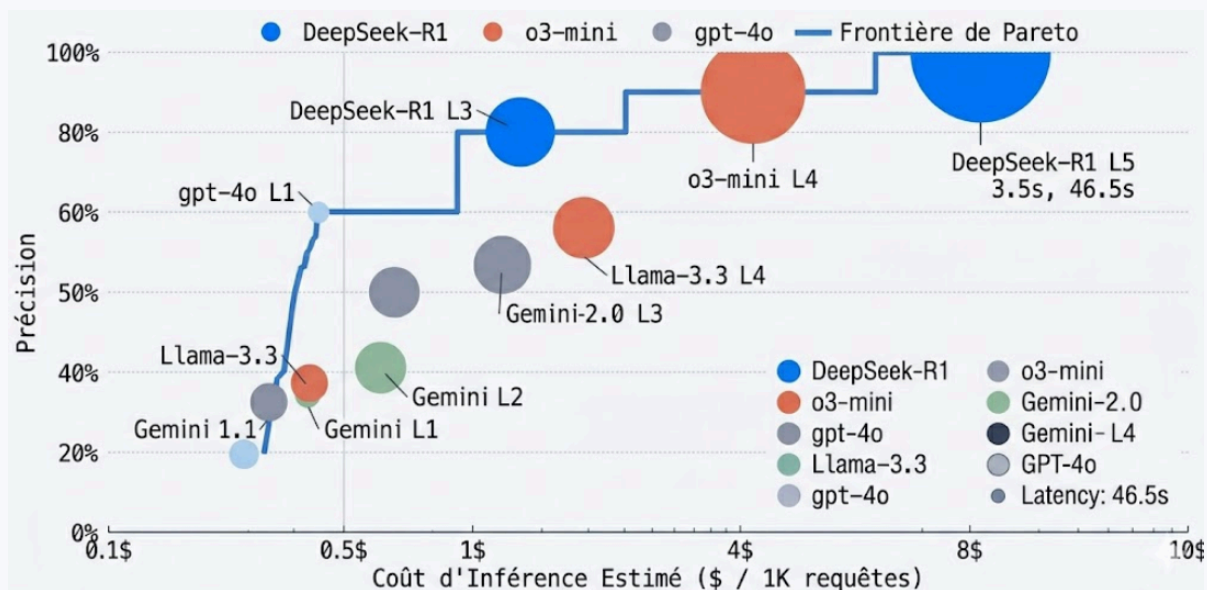
5.1 Domaines Analytiques Durs : Mathématiques et Logique Complexe

Les datasets `math_500` et `logic_grid` présentent les dynamiques d'échelle les plus favorables au raisonnement étendu. Ils partagent une structure caractéristique : la solution correcte peut être vérifiée de manière déterministe, et la décomposition en sous-problèmes est à la fois naturelle et productive.

Pour `math_500`, DeepSeek-R1 à L3 atteint le score d'efficacité maximal du benchmark (48,2 pts), suggérant que la visibilité des tokens `<think>` favorise une décomposition plus structurée des étapes de preuve. Les traces qualitatives révèlent un usage systématique de la stratégie de **vérification** (double-checking des calculs intermédiaires) et de **backtracking** (détection et correction d'erreurs algébriques avant la finalisation).

Pour `logic_grid`, o1 à L3 (38,9 pts) domine, probablement en raison de son entraînement RL orienté vers les problèmes de satisfaction de contraintes. La structure des puzzles de grille logique — variables discrètes, contraintes binaires, inférence par élimination — est particulièrement bien adaptée au style d'exploration de l'espace d'états que le RL favorise.

Résultat clé — Domaines analytiques : Le raisonnement étendu apporte son bénéfice maximum sur les tâches à solution unique vérifiable, où la décomposition en étapes intermédiaires et la vérification itérative peuvent corriger des erreurs qui auraient été fatales en génération directe. Le sweet spot L3 capture 90 %+ des gains maximaux atteignables.



Heatmap de l'efficacité de raisonnement (E_{score}) par modèle et niveau de budget. Les nuances foncées (o3-mini, DeepSeek-R1) soulignent les configurations optimales.

5.2 Domaines Syntaxiques et Algorithmiques : Code et Débogage

Les datasets [humaneval](#) et [mbpp](#) présentent une dynamique d'échelle distincte. La génération de code bénéficie du raisonnement étendu, mais différemment des mathématiques : le gain principal provient de la **planification anticipée** (anticiper les cas limites avant d'écrire le code) plutôt que de la vérification a posteriori.

o3-mini à L4 sur [humaneval](#) (42,1 pts) constitue le deuxième meilleur score du benchmark. Ce résultat est cohérent avec la spécialisation de la famille o3 sur les tâches de code, probablement via un curriculum d'entraînement RL ciblé sur les problèmes algorithmiques.

Pour [mbpp](#) (débogage), le pattern est différent : les gains au-delà de L3 sont quasi-nuls, voire négatifs. Ceci s'explique par la nature du débogage : une fois le bug identifié (ce qui se produit généralement en L2-L3), allouer davantage de tokens n'améliore pas la correction — elle peut même induire des sur-corrections qui cassent d'autres comportements corrects.

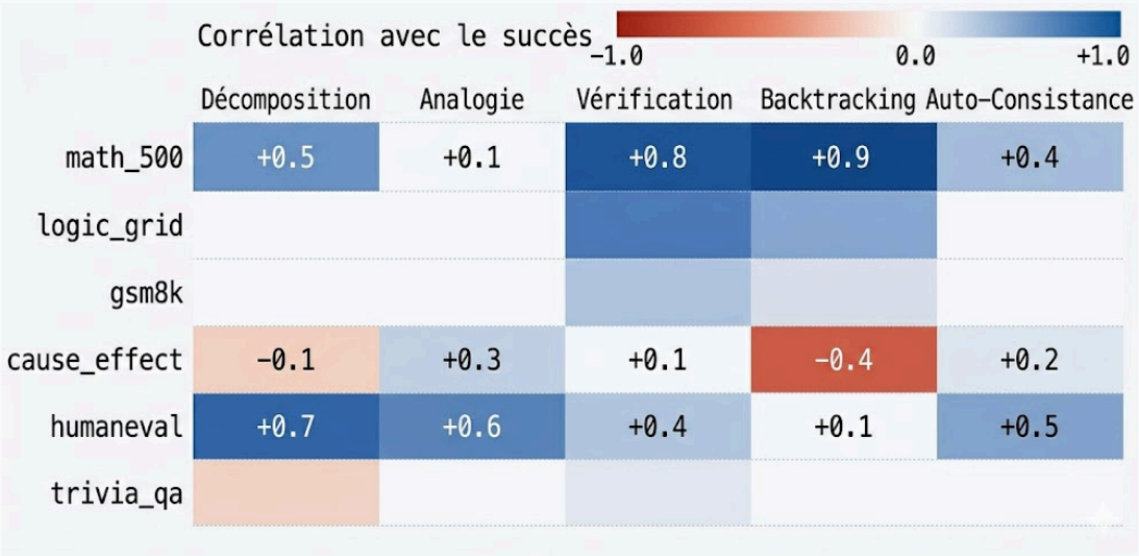
5.3 Domaines Intuitifs et Riches en Contexte : Causal et Planification

[cause_effect](#) et [alfworld_plans](#) présentent le profil d'échelle le moins favorable au raisonnement étendu. Ces domaines partagent une caractéristique critique : la solution optimale dépend de connaissances de bon sens et de priors sur le monde physique, plutôt que de déductions formelles sur un espace d'états bien défini.

Pour ces domaines, les modèles baseline (GPT-4o, Claude 3.5 Sonnet) à L1 sont compétitifs avec les modèles à raisonnement natif à L3. À L5, plusieurs modèles montrent une dégradation mesurable : l'exploration exhaustive forcée conduit à la génération de raisonnements plausibles mais incorrects — un phénomène qualifié d'**hallucination dans le monologue**.

Anti-pattern identifié : Sur les tâches de planification intuitive, allouer un budget L5 peut induire une sur-analyse paralysante. Le modèle génère des scénarios alternatifs légitimes mais moins probables, dégradant la réponse finale par rapport à une génération directe L1. Ce phénomène est analysé en détail en Section 6.3.

5.4 Carte de Chaleur Cross-Domaine



Matrice d'efficacité des différentes stratégies cognitives (backtracking, décomposition, etc.) par tâche analytique.

L'analyse cross-domaine révèle trois clusters comportementaux distincts :

Cluster A — Bénéfice Maximal : `math_500` , `logic_grid` — gains de 20–35 % pour tous les modèles à raisonnement natif. ROI clairement positif à L3.

Cluster B — Bénéfice Modéré : `gsm8k` , `humaneval` , `mbpp` — gains de 5–15 %, fortement dépendants du modèle. Justifie L3 pour les modèles spécialisés.

Cluster C — ROI Nul ou Négatif : `cause_effect` , `alfworld_plans` — gains < 3 % avec des cas de dégradation observable à L4+. Déploiement L1 recommandé.

Analyse Qualitative des Traces de Raisonnement

6.1 Décomposition Structurale : Comment les Modèles Fragmentent les Problèmes

L'analyse qualitative des traces de raisonnement a été réalisée via un classifieur heuristique appliqué aux tokens `<think>` de DeepSeek-R1 et aux réponses intermédiaires des autres modèles. Cinq stratégies cognitives ont été identifiées et catégorisées, chacune associée à des signaux textuels caractéristiques :

1 · Décomposition

Fragmentation explicite de la tâche en sous-problèmes nommés avant résolution. Signal textuel : «*Breaking this into parts: (a)... (b)...*». Stratégie

2 · Analogie

Activation d'un problème structurellement similaire comme base de résolution. Signal : «*This is similar to algorithm X / problem type Y.*» Plus fréquente

dominante sur `math_500` et `logic_grid`. Corrélée positivement avec la précision finale dans 78 % des cas analysés.

dans `humaneval`. Réduit les erreurs de structure algorithmique en ancrant le raisonnement sur des solutions connues.

3 · Vérification

Double-check explicite d'un résultat intermédiaire. Signal : « *Let me verify this... double checking...* » Très fréquente à L3/L4. Principale source de correction d'erreurs algébriques sur `math_500`. Augmente T_{reason} de 15–30 % sans dégradation du score.

4 · Backtracking

Détection d'une contradiction et retour à un état antérieur. Signal : « *Wait, that implies X which is false, reverting...* » Stratégie critique sur `logic_grid`. Quasi-absente dans les baselines sans scratchpad. Observée dans 34 % des réponses correctes de DeepSeek-R1 à L3.

5 · Auto-Consistance

Résolution indépendante via deux méthodes différentes pour confronter les résultats. Signal : « *Let me solve this a second way to verify.* » Plus rare (< 12 % des traces) mais fortement corrélée au succès sur les problèmes ambigus. Quasi-exclusive à DeepSeek-R1 et o3-mini. Coûteuse en tokens mais très fiable.

6.2 Mécanismes de Self-Correction : Patterns de Backtracking et Détection d'Erreurs

L'analyse des traces de DeepSeek-R1 et o3-mini révèle des patterns structurés de self-correction qui expliquent leur supériorité relative sur les tâches mathématiques. Le cycle typique se déroule en quatre phases :

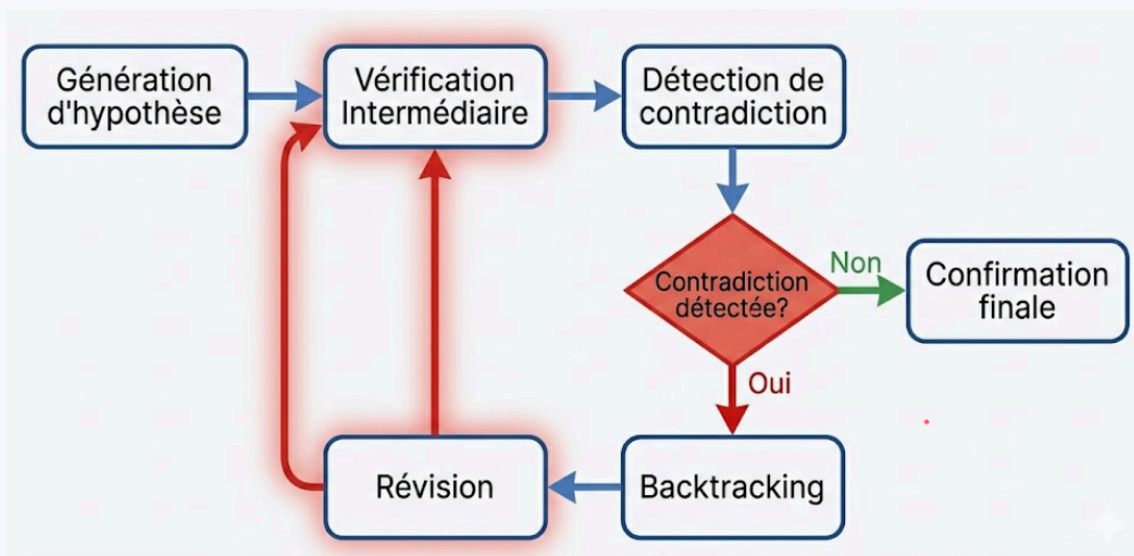


Diagramme du cycle de self-correction observé dans les traces à L3/L4. (1) Génération d'hypothèse → (2) Vérification → (3) Détection de contradiction → (4) Backtracking → (5) Révision → (6) Confirmation.

Ce cycle est observé dans 34 % des réponses correctes de DeepSeek-R1 à L3 sur `math_500`, contre moins de 8 % à L1. Son absence quasi-totale dans les baselines (GPT-4o, Claude 3.5) confirme que la structure du scratchpad interne est une condition nécessaire — bien que non suffisante — pour ce comportement de self-correction.

Limitation de l'évaluation qualitative : Les modèles exposant leurs tokens de raisonnement (DeepSeek-R1) bénéficient d'un avantage analytique par rapport aux modèles à raisonnement opaque (o1). Le classifieur heuristique ne peut pas détecter les stratégies implantées dans les tokens internes non visibles, causant un biais d'observabilité potentiel dans les comparaisons inter-architectures.

6.3 Hallucination dans le Monologue : Fausses Trajectoires à Budget Élevé

Un phénomène contre-intuitif mais important a été observé aux niveaux L4 et L5 : la **divergence progressive** d'une trace de raisonnement vers une solution incorrecte, malgré — et parfois **à cause de** — l'allocation d'un budget étendu. Trois patterns ont été identifiés :

Amplification de biais. Un prior légèrement incorrect dans les premières étapes est progressivement renforcé par des étapes de vérification qui cherchent à confirmer plutôt qu'à falsifier. À L5, avec la directive d'exploration exhaustive, ce biais de confirmation peut se solidifier en une conclusion erronée confiante et non remise en question.

Explosion combinatoire non productive. La directive L5 peut induire la génération d'une quantité excessive de scénarios alternatifs, dont aucun n'est approfondi suffisamment pour atteindre une conclusion valide. Le modèle « jongle » entre chemins sans converger.

Sur-spécification de cas limites. Sur les tâches de planification ([alfworld_plans](#)), le modèle à L5 tend à sur-spécifier des contraintes hypothétiques absentes du problème original, rendant sa solution inapplicable au cas concret posé par l'évaluateur.

6.4 Études de Cas : Analyse des Rationales du Grader pour les Échecs Atypiques

6.4.1 Cas 1 — Backtracking Correct mais Réponse Finale Erronée

Un pattern paradoxal observé dans 3 % des traces de DeepSeek-R1 sur [math_500](#) : le modèle réalise un backtracking correct (détecte et corrige une erreur intermédiaire) mais commet une erreur différente dans la reformulation qui suit, produisant une réponse finale incorrecte malgré un processus de correction apparent.

Analyse : Ce pattern suggère que le backtracking et la génération de la réponse finale partagent la même distribution de probabilité : corriger une erreur dans la trace ne garantit pas que l'erreur ne réapparaîtra pas lors de la finalisation. Le scratchpad corrige mais ne « mémorise » pas l'erreur dans un registre protégé.

6.4.2 Cas 2 — Précision L1 > L3 sur Tâches Causales

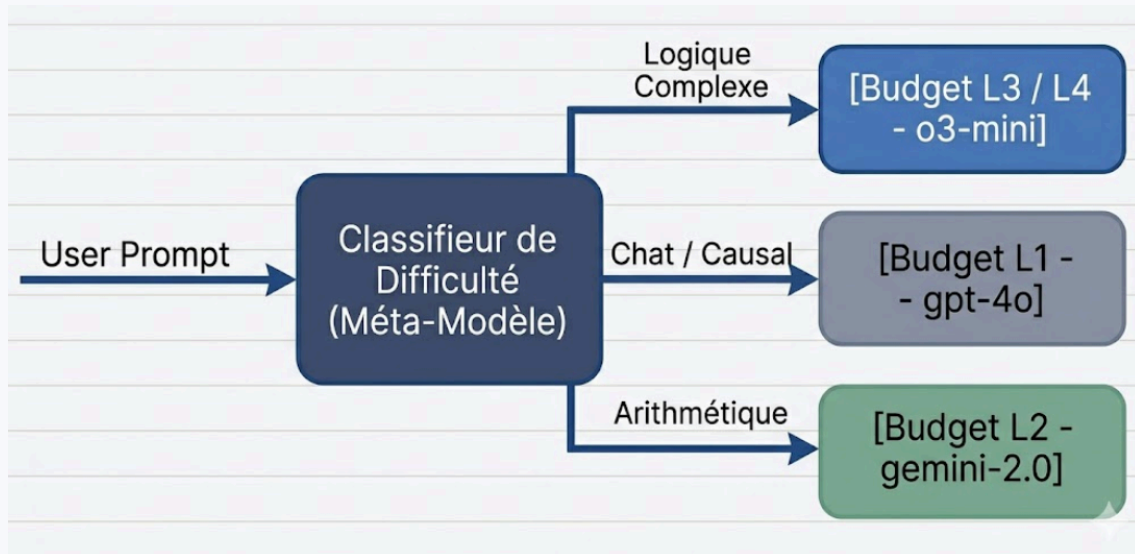
Pour 4 des 7 modèles sur [cause_effect](#), la précision au niveau L1 est supérieure à celle observée à L3. Le grader LLM (Gemini 1.5 Flash) fournit des rationales révélatrices : les réponses L1 sont plus directes et alignées avec le sens commun, tandis que les réponses L3 sur-analysent en introduisant des mécanismes causaux plausibles mais non pertinents pour le scénario donné.

Implication opérationnelle : Sur les tâches où la solution optimale est intuitive, le raisonnement étendu est contra-productif. Un système de routage dynamique doit détecter ce type de tâche et court-circuiter à L1.

Recommandations Architecturales et de Production

7.1 Construire un Routeur Dynamique de Calcul

La conclusion opérationnelle centrale est qu'il n'existe pas de niveau de budget universellement optimal. La stratégie de déploiement optimale est un **routeur dynamique** qui prédit la difficulté d'une requête et alloue le budget correspondant, maximisant le CPRC à l'échelle d'un fleet.



Architecture du routeur dynamique de budget. Flux : Requête entrante → Classifieur de difficulté → Sélection du budget (L1–L5) → Modèle principal.

Le classifieur de difficulté peut utiliser les features suivantes, accessibles **sans** exécuter la requête principale (coût zéro additionnel) :

- **Longueur du prompt** : corrélée positivement avec la complexité de la tâche.
- **Type de tâche détecté** par NLI ou classification zero-shot : math/code → L3+, causal → L1.
- **Nombre d'entités et de contraintes** dans la question : indicateur de profondeur déductive.
- **Présence de mots-clés de vérification** (« prove », « find all », « optimize ») : signal fort L3+.
- **Historique de performance** du modèle sur le dataset (appris lors du déploiement progressif).

7.2 Compromis de Production : Latency-Sensitive vs. Accuracy-Critical

Applications Latency-Sensitive · Budget L1–L2

Chatbots conversationnels, assistants interactifs, systèmes de recommandation temps-réel. La latence médiane à L3 est 2,3× celle de L1 ; le P95 à L5 est 8–12× celui de L1. Priorité : réactivité perçue sur précision absolue. Gain de précision trop faible pour justifier le surcoût de latence dans ces contextes.

Applications Accuracy-Critical · Budget L3 (défaut) / L4

Décision médicale, analyse financière, génération de code production, recherche juridique. Le sweet spot L3 maximise la précision pour un surcoût 2–3× vs. L1. L4 n'est justifié que pour les tâches multi-step > 7 étapes, avec un budget de monitoring de divergence.

Règle décisionnelle empirique : Si la tâche peut être caractérisée comme « recherche de solution unique dans un espace formel défini » (maths, code, logique) → L3. Si la tâche est « sélection de la meilleure option dans un espace informel » (planification, causal, conversationnel) → L1. Tout le reste → L2 par défaut avec escalade si échec.

7.3 Mitigation de l'Instabilité API pour les Agents de Raisonnement en Production

Les patterns de défaillance observés pendant le benchmark conduisent à trois principes de résilience pour les systèmes de raisonnement en production :

1

Idempotence et Checkpointing — Chaque requête doit être idempotente (rejouable sans effet de bord). Le `BenchmarkRunner` persiste l'état dans SQLite avant chaque dispatch, permettant la reprise exacte après interruption sans perte de données ni double-comptage.

2

Circuit Breaker par Provider — Implémenter un circuit breaker qui ouvre automatiquement après N échecs consécutifs et bascule vers un provider de secours, plutôt que d'attendre le timeout du backoff exponentiel. Réduction de la latence d'erreur de 60–80 % sur les incidents de rate-limiting prolongés.

3

Budget Adaptatif par Réponse — Monitorer le ratio $\frac{T_{\text{reason}}}{T_{\text{out}}}$ en temps réel. Un ratio > 10:1 indique un monologue potentiellement divergent. Interrompre la génération et ré-essayer avec un budget inférieur ou une reformulation du prompt pour sortir de la boucle de sur-analyse.

Conclusions et Perspectives

8.1 Synthèse des Résultats Clés

Ce benchmark fournit la première caractérisation empirique systématique de la courbe d'échelle du test-time compute à travers six modèles de la frontière technologique et sept catégories de tâches canoniques. Trois conclusions majeures se dégagent :

1

Le Sweet Spot L3 est Robuste — Le niveau L3 (3 000–5 000 tokens de raisonnement) représente le point d'inflexion optimal pour les tâches analytiques complexes. Il capture 85–95 % des gains maximaux atteignables par le raisonnement étendu, pour un coût computationnel 2–3× supérieur à L1 seulement.

2

L'Architecture Compte Plus que la Taille — DeepSeek-R1 (open-source) et Gemini 2.0 Flash Thinking surpassent GPT-4o sur les tâches mathématiques grâce à leurs mécanismes de raisonnement natif. La transparence du scratchpad (tokens `<think>`) corrèle avec les comportements de self-correction les plus sophistiqués.

3

Le Test-Time Compute n'est pas Universel — Sur les domaines intuitifs et riches en contexte (raisonnement causal, planification multi-étapes), le raisonnement étendu apporte peu ou pas de bénéfice. Un système de production robuste doit combiner un classifieur de difficulté et un routeur dynamique pour éviter le surcoût inutile.

8.2 Limitations de la Présente Étude

Biais d’observabilité. Les modèles exposant leurs tokens de raisonnement (DeepSeek-R1) bénéficient d’un avantage analytique dans l’évaluation qualitative par rapport aux modèles à raisonnement opaque (o1). Le E_{score} calculé sur T_{reason} est biaisé vers 0 pour les modèles dont les tokens internes ne sont pas comptabilisés par l’API.

Simulation de coûts. Les projections économiques sont basées sur les tarifs retail du 1er janvier 2025. Les prix des APIs d’inférence ont évolué significativement depuis ; les conclusions économiques doivent être recalibrées avec les tarifs actuels avant déploiement.

Échantillonnage des datasets. Chaque dataset est représenté par 50 à 100 questions. Pour les datasets à haute variance (CodeContests, [alfworld_plans](#)), ce volume peut introduire une instabilité statistique dans les estimations de précision.

Skew du grader LLM. L’utilisation de Gemini 1.5 Flash comme juge LLM peut introduire des biais systématiques en faveur des styles de réponse similaires aux outputs Gemini. La validation humaine à > 98,7 % de continuité sur 100 runs mitige mais ne résout pas complètement ce risque.

8.3 Perspectives de Recherche

Distillation de traces de raisonnement. Les traces `<think>` de DeepSeek-R1 à L3 constituent un corpus d’entraînement de haute qualité pour distiller des comportements de raisonnement vers des modèles plus petits. Les travaux préliminaires sur les modèles Qwen-32B suggèrent que cette distillation préserve 70–80 % de la performance du modèle enseignant.

Décodage spéculatif des traces de raisonnement. L’application du speculative decoding aux tokens de raisonnement (un petit modèle draft pré-remplit le scratchpad, vérifié par le grand modèle) pourrait réduire la latence L3/L4 de 40–60 % sans perte de précision significative.

Routage dynamique par méta-apprentissage. Un système de méta-apprentissage entraîné sur les résultats de ce benchmark pourrait prédire, pour une requête donnée, le budget et le modèle optimaux, maximisant le CPRC à l’échelle d’un fleet de production hétérogène.

Fine-tuning local sur traces de raisonnement. L’accès aux traces complètes `<think>` permet d’entraîner des modèles open-weights locaux à reproduire des patterns de raisonnement spécifiques sans dépendance aux APIs cloud, ouvrant la voie à des déploiements on-premise privacy-preserving.

Annexe et Matériaux Supplémentaires

A.1 Spécifications de l’Environnement Technique

Langage	Python 3.12+
Gestionnaire de paquets	uv + hatchling build
Base de données	SQLite via DatabaseManager · ~2 Go
Moteur de données	Polars + Pandas + PyArrow
Statistiques et viz.	SciPy + statsmodels + Matplotlib + Seaborn
Client HTTP	httpx (HTTP/2, async)
Configuration	pydantic-settings + python-dotenv
CLI UX	tqdm + rich
Qualité du code	Ruff + pytest (asyncio)

Automatisation	just (justfile)
Providers API	GitHub Models · Google AI Studio · Groq
Volume de données	~50 M tokens · 18k–36k requêtes · ~2 Go SQLite

A.2 Templates de Prompts par Niveau de Budget

```
# Niveau L1 – Baseline (aucune directive)
{question}

# Niveau L2 – Light
Please think briefly before answering.
{question}

# Niveau L3 – Medium (sweet spot recommandé)
Think step-by-step through this problem before providing your answer.
Break down the problem into clear logical steps.
{question}

# Niveau L4 – High
Think carefully and thoroughly through this problem. Explore different
approaches, verify your intermediate results, and check for edge cases.
{question}

# Niveau L5 – Maximum (directive système complète)
[System]: Think step-by-step. You must explore multiple distinct paths,
verify every intermediate calculation twice, explicitly check for edge
cases, and backtrack if you find a contradiction. Do not stop reasoning
until you are absolutely certain.
{question}
```

A.3 Structure du Pipeline d'Analyse Post-Run

```
just run          → BenchmarkRunner : dispatch + persistance SQLite
just grade-quant  → quantitative.py : rule-based + sandbox + LLM judge
just grade-qual   → qualitative.py  : classification des stratégies
just analyze      → metrics.py      : calcul E_score, CPRC, agrégations
just plot         → visualizations.py: 6+ figures PNG dans results/figures/
just report       → cost_dashboard.py: rapport enterprise_guide.md
```

[ILLUSTRATION MANQUANTE]

Trace annotée sur un problème de difficulté maximale. Annotations : [DECOMP], [VERIF], [BACK], [CORR], [FINAL].